

lec 54

STACK

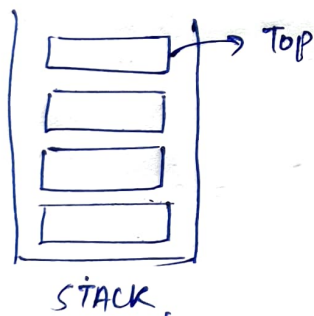
→ follows LIFO (last in first out)

* operations

- 1) PUSH
- 2) POP
- 3) PEEK (top element shown)
- 4) empty

syntax: using STL;

```
stack<int> s;
s.push(2);
s.pop();
s.top();
s.empty(); → True
           → false
```



stack using class (Array)

```
class Stack {
public: int *arr;
       int top;
       int size;
```

```
Stack (int size) {
```

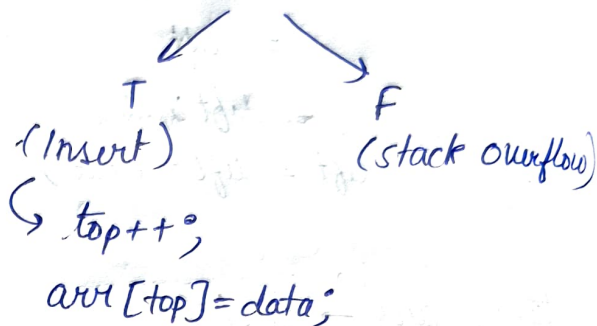
```
    this->size = size;
    arr = new int[size];
    top = -1;
```



top = -1

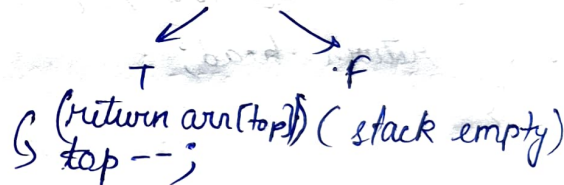
o) push Operation.

check
(space available)?



o) Pop Operation.

check
(top ≥ 0)



```
void push (int d) {
    if (size - top > 1) {
        top++;
        arr[top] = element;
    } else {
        cout << "stack overflow";
    }
}
```

```

void pop() {
    if (top >= 0) {
        top--;
    } else {
        cout << "Stack Empty";
    }
}

```

```

int peek() {
    if (top >= 0 && top < size) {
        return arr[top];
    } else {
        cout << "top not in range";
    }
}

```

```

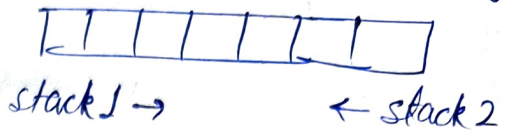
bool is Empty() {
    if (top == -1) {
        return true;
    } else {
        return false;
    }
}

```

All functions $O(1)$

Q.) Two stack in 1 Array.

Soln



fill till full array is filled.

MC 55

Q.) Reverse string using STACK

① push all elements (1 to n)

② Pop all. T.C $O(n)$ S.C. $O(n)$

Q.) Delete middle element from stack.

```

void deleteMiddle(stack& s, int n) {
    int size = s.size();
    int count = 0;
    if (count == size/2) {
        s.pop();
        return;
    }
    // base case
}

```

```

void solve(stack s, int cnt,
            int n) {

```

// base case

```

if (size == cnt/2) {
    s.pop();
    return;
}

```

```

int num = s.top();
s.pop();

```

```

solve(s, cnt+1, n);

```



```

        s.push(num);
    }

void deleteMiddle (stack s, int n)
{
    int count = 0;
    solve (s, count, n);
}

```

Q.) Check valid Parenthesis

[{()}] ✓ balanced.

{()} × not balanced.

```

bool isValidP (string exp) {
    stack s;

```

```

for (int i = 0; i < exp.size(); i++) {
    for (int i = 0; i < exp.size(); i++) {

```

```

        char ch = exp[i];

```

```

        if (ch == '(' || ch == '{' ||
            ch == '[') {

```

```

            s.push(ch);
        } else {

```

```

            if (!s.empty()) {

```

```

                char top = s.top();

```

```

                if (matches (top, ch)) {
                    s.pop();
                } else {

```

```

                    return false;
                }
            } else {

```

Q.) Curtain .

```

        return false;
    }

    if (s.empty()) {
        return true;
    } else {
        return false;
    }

```

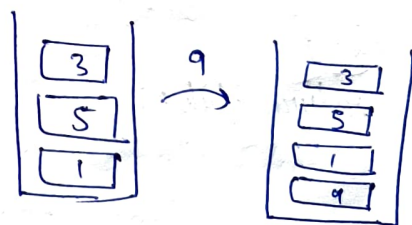
// replace in place of matches()

```

    if (ch == '(' && top == '(' ||
        ch == '[' && top == '[' ||
        ch == '}' && top == '{')

```

Q.) Insert at bottom of stack



recursion

// base

```

if (empty) {
    s.push(x);
    return
}

```

```

int num = s.top();
s.pop();

```

```

solve (s, x);
s.push (num);
}

```

T.C = $O(n)$ S.C = $O(1)$

Q.) Reverse A stack

- ① store first element (top)
- ② recursion backe hue ko reverse karke layega.
- ③ Insert at bottom of reversed stack

```

void reverseStack (stack s) {
    // base
    if (stack.empty()) {
        return;
    }
    int num = s.top();
    s.pop();
    reverseStack (s);
    insert At bottom (s, num);
}

```

Q.) Sort Stack

Basically insertion sort.

```

void sortStack (stack<int> s) {
    // base
    if (s.empty()) {
        return;
    }
}

```

```

int num = s.top();
s.pop();

```

// recursive call.

sortStack (s);

sorted Insert (s, num);

```

void sortedInsert (s, num) {
    if (s.empty() || (!s.empty()
        && s.top < num)) {
        s.push (num);
        return;
    }
}

```

```

int n = s.top();
stack.pop();

```

```

// recursive step
sortedInsert (s, num);
stack s.push (n);
}

```

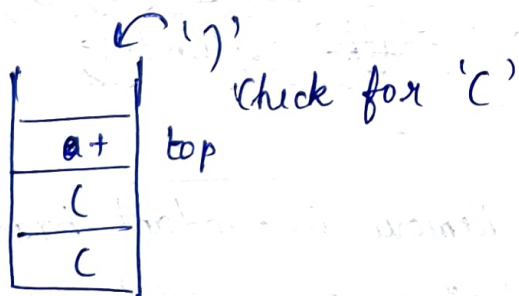
Q.) Remove Redundant Brackets.

→ logic : agar brackets ke bich mai operators hai then wo brackets useful hai.

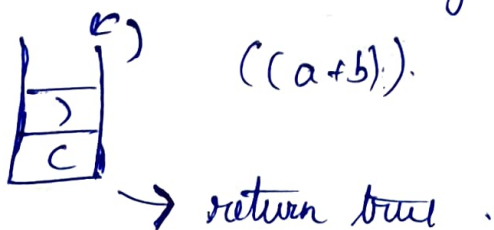
Approach

- ① Push opening brackets & operators (+, -, *, /)
- ② if closing bracket is readed go back until you find its opening bracket
- ③ If you find any operator before opening bracket it means it is NOT a redundant bracket and pop till that bracket
- ④ If no operator found, return true (there is redundant bracket)

e.g. $\rightarrow ((a+b))$



Since '+' is present pop all elements till you get one '(' (including '(')



Q. minimum cost to make str valid

$\{\{\}$ $\rightarrow -1 \rightarrow$ impossible

$\{\{\{\}$ $\rightarrow 2$

$\{\}\}\}$ $\rightarrow 2$

$\}\}\{\}$ $\rightarrow 2$

Algo.

① If str.size = odd $\rightarrow -1$

② remove valid part

$\{\}\}\}$ $\rightarrow \{\}$

$\{\}\}\}\{\}$ $\rightarrow \underbrace{\{\}\}_{a} \underbrace{\{\}\}_{b}$

③ closing = a
opening = b

$$\text{ans} = \left(\frac{a+1}{2}\right) + \left(\frac{b+1}{2}\right)$$

Q Lec 56

Next Small Element.
using stack

i/p

2	3	1	4
---	---	---	---

o/p

1	1	-1	-1
---	---	----	----

approach

$[2, 1, 4, 3]$
start from behind



③
Chota element
lao

s.top()

3 se chotta

→ ans store

→ s.push(ans)

3 se bada

Chota
dhund
lao

while

(s.top
element >= cur)

{
s.pop();
}

ans = s.top()

s.push(ans);

② ① ④ ③



1 1 3 1

~~1~~ 4 → pop

3 > 1

4 > 3

~~1~~ 4 → pop

1 > 3 → pop

1 > 1 → push

Q.) largest area in histogram

$$\text{area} = l \times b$$

→ maximise

• length is fixed

• we can extend left & right



→ ④ ans

find prev chota index &
next chota index

$$\text{max width} = n - p - 1$$

Q.) Celebrity Problem. (lec 56)

→ Everyone knows celebrity

→ Celebrity knows no-one.

	1	2	3
1	0	1	0
2	0	0	0
3	1	1	0

$a_{ij} \rightarrow i \text{ knows } j \text{ if } a_{ij} = 1$

return celebrity

here, ②

Brute force

- 1) celebrity rows = 0
- 2) celebrity column = 1 ($a_{ii} = 0$)

approach 2

- ① Put all people in stack
- ② take two persons at a time and check each other,
if ($a_{ij} = 1$ remove i)
($a_{ji} = 1$ remove j)
- ③ ~~pop~~ push possible celebrity i
if $a_{ij} = 0$
- ③ Pop next 2 and check till
 $S.size = 1$;

$TC = O(N)$

Q. Max rectangle in binary matrix with all 1's.

e.g. ans = 8

	0	1	2	3
0	0	1	1	0
1	1	1	1	1
2	1	1	1	1
3	1	1	0	0

approach.

Same as largest area in histogram
But go row-wise & find extension in height.

- ① Compute max area for 1st row
- ② For every remaining rows add upper wall row in cur row (ie i th row).
- ③ Compute Area.

T.C $O(m \times n)$ SC $O(m)$

Q. 'N' stacks in an array. lec 5b

approach 1

divide array into 'k' parts and assign 1 stack to each part.

↳ drawback (space not used in optimised way).

approach 2

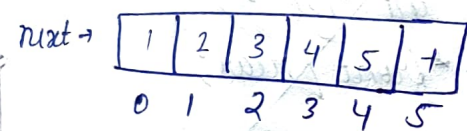
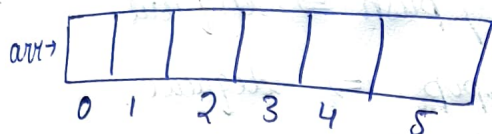
• $top[] \rightarrow$ This array represents index of top elements.

• $next[] \rightarrow$ 1) Point to next element after stack top
2) point to next free space.

$N \rightarrow$ no. of stacks = 3

$s \rightarrow$ size of array = 6

$Q \rightarrow$ no. of queries = 5



Queries - 1) push (10, 1)
push '10' in 1st stack
2) push (20, 1)
3) push (30, 2)
4) pop (1)
5) pop (2)

int freeSpot = 0;
push (10, 1)

I) find freeSpot

int index = freeSpot;

II) Update freeSpot;

freeSpot = next[freeSpot];

III) Insert in array

arr[index] = x;

IV) Update next;

next[index] = top[m-1];

V) Update top

top[m-1] = index;

for pop(m) mth stack
check underflow
if (top[m-1] == -1) {
return false
}

int index = top[m-1];

top[m-1] = next[index];

next[index] = freeSpot;

freeSpot = index;

return arr[index];

}

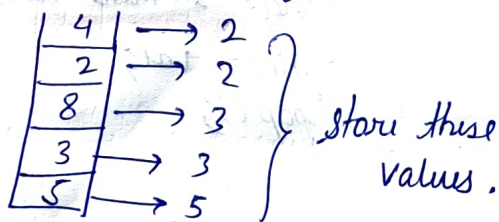
T.C = $O(1)$

S.C = $O(s+n)$

Q. Special Stack Problem.

$\rightarrow$ Design a stack that supports
getMin() in $O(1)$ time & $O(1)$ space

approach 1 $\rightarrow O(n)$ space
 $\rightarrow T(1)$ time



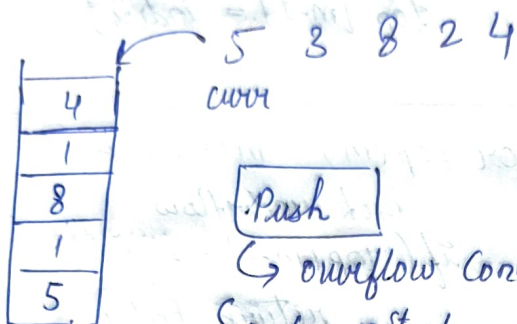
push() $\rightarrow$ I) normal push

II) min value $\rightarrow$ IInd stack

pop() $\rightarrow$ I) pop from both.

getMin() $\rightarrow$ I) return 2nd stack top.

approach 2



Push

→ overflow Condⁿ
→ for 1st element
→ normal push
→ mini update
→ back^o elements
→ if (curr < mini){
 val = $2 * \text{curr} - \text{mini}$
 push(val);
 mini update;
} else {
 normal push
}

POP

→ underflow Condⁿ
→ if (curr > mini)
 normal pop();

else

val = $2 * \text{mini} - \text{curr}$
mini = val;
pop();

getmin()

→ mini stored
→ return;

lc60

QUEUE

FIFO - first in first out.

Implementation using array

insert / push ⇒ enqueue

pop ⇒ dequeue

~~Class Queue {~~

~~int arr[];~~

~~int size;~~

~~int *front / rear;~~

~~}~~

~~void enqueue (int data) {~~

~~if (rear~~

Class Queue {

int *arr;

int front;

int rear;

int size;

}

public :

~~void~~ Queue() {

size = 100001;

arr = new int[size];

front = 0;

rear = 0;

}

void enqueue (int d)

if (rear == size) {
 cout << "queue is full";
}

```

else {
    arr[rear] = data;
    rear++;
}

```

```

int dequeue() {
    if (front == rear) {
        // queue is empty
        return -1;
    } else {
        int ans = arr[front];
        arr[front] = -1;
        front++;
        return ans;
    }
}

```

```

int front() {
    if (front == rear) {
        return -1;
    } else {
        return arr[front];
    }
}

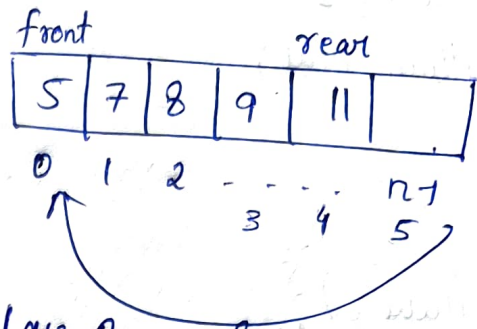
```

```

bool isEmpty() {
    if (front == rear) {
        return true;
    } else {
        return false;
    }
}

```

Circular Queue



Class Queue {

```

    int *arr;
    int size;
    int front;
    int rear;

```

public :

```

    Queue (int n) {
        size = n;
        arr = new int[size];
        front = -1;
        rear = -1;
    }

```

```

bool enqueue (int value) {
    // check full
    if (front == 0 && rear == size-1) {
        // rear = (front-1) % (size-1)
        return false;
    } else if (front == -1) // first element {
        front = rear = 0;
    } else if (rear == size-1 && front != 0) {
        rear = 0;
    } else {
        rear++;
    }
}

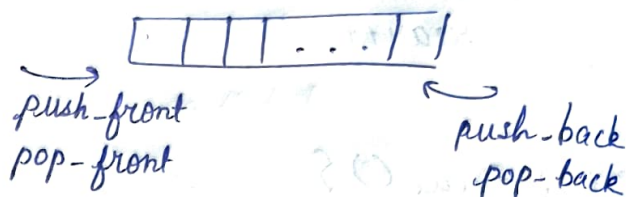
```



```
arr[rear] = value;
return true;
}
```

```
int dequeue() {
    // Check empty
    if (front == -1) {
        return -1;
    }
    // for single element
    if (front == rear) {
        int ans = arr[front];
        if (front == rear) {
            arr[front] = -1;
            front = -1;
            rear = -1;
        }
        else if (front == size-1) {
            front = 0;
        }
        else {
            front++;
        }
        return ans;
    }
};
```

Doubly Ended Queue.



```
front = -1
push rear = -1
```

STL:

```
deque<int> d;
```

```
d.push_front(12);
d.push_back(14);
d.pop_front();
```

```
[12 | 14]
```

```
d.empty();
```

Implementation

- same code

Order

- 1) CHECK full or empty
- 2) Check first or single element
- 3) maintain cyclic Nature
- 4) Normal flow: front++, rear++;

Q. Queue Reversal

lec 61

i/p

4	6	1	3	5
---	---	---	---	---

o/p

5	3	1	6	4
---	---	---	---	---

approach 1

→ using stack

T.C = $O(n)$

S.C = $O(n)$

approach 2

→ recursion

Q. First negative integer in every window of size 'k'

i/p

-8	2	3	-6	10
----	---	---	----	----

k=2

o/p

-8	0	-6	-6
----	---	----	----

code

```
vector<int> print(int A[], int n, int k) {
```

```
    deque<int> dq;
```

```
    vector<int> ans;
```

```
    // process first window
```

```
    for (int i=0; i<k; i++) {
```

```
        if (A[i] < 0) {
```

```
            dq.push_back(i);
```

```
        }
```

```
    // store ans for first window
```

```
    if (dq.empty()) {
```

```
        ans.push_back(0);
```

```
    } else {
```

```
        ans.push_back(A[dq.front()]);
```

```
    // move window ahead
```

```
    for (int i=k; i<N; i++) {
```

```
        // Removal
```

```
        if (!dq.empty() &&
```

```
            i - dq.front() >= k) {
```

```
            dq.pop_front();
```

```
        // addition
```

```
        if (A[i] < 0) {
```

```
            // deque
```

```
            dq.push_back(i);
```

```
    // ans store
```

```
    if (dq.size() > 0) {
```

```
        ans.push_back(A[dq.front()]);
```

```
    } else {
```

```
        ans.push_back(0);
```

```
    }
```

```
}
```

```
return ans;
```

```
}
```


- Logic
- 1) First window process
 - 2) Store all indices of arr which contains negative no. in deque.
 - 3) Process further window by adding & removing from deque.

Q. Reverse first k elements of queue

i/p $\rightarrow$

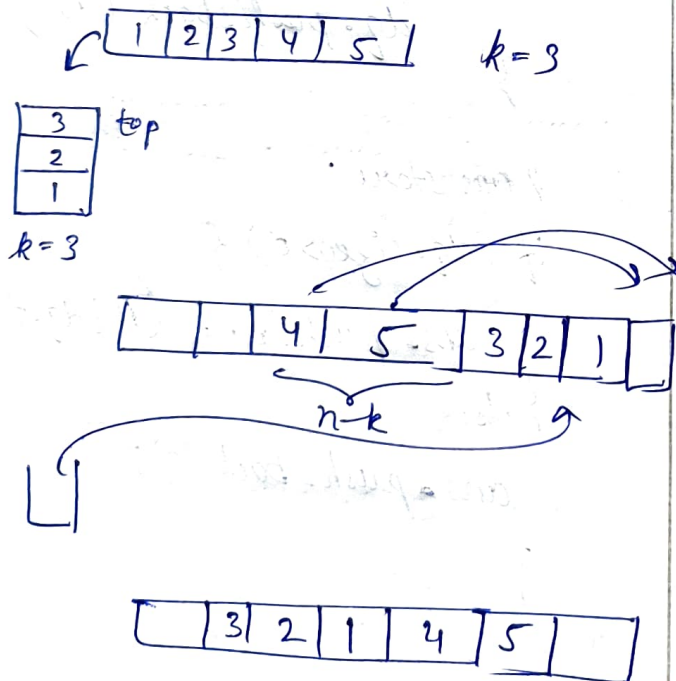
1	2	3	4	5
---	---	---	---	---

 $k=3$

o/p

3	2	1	4	5
---	---	---	---	---

Logic: use stack of size ' k '



Q. First non-repeating character in a stream.

i/p - aabc

o/p - a#bb

explanation:

each time 1 lower case char is inserted in stream.

Return non-repeating character of stream each time.

1) "a" $\rightarrow$ "a"

2) "aa" $\rightarrow$ "#"

3) "aab" $\rightarrow$ "b"

4) "aabc" $\rightarrow$ "b"

a#bb ans

approach

FirstNonRepeating (string A) {

map < char, int > count;

string ans = " ";

queue < char > q;

for (int i=0; i < A.size(); i++) {

char ch = A[i];

q.push(ch);

count[ch]++;

while (!q.empty()) {

if (count[q.front()] > 1) {

q.pop();

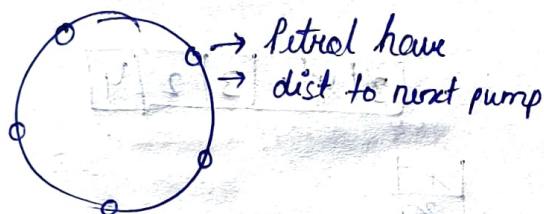
} else {

```
ans.push_back (q.front());
break;
```

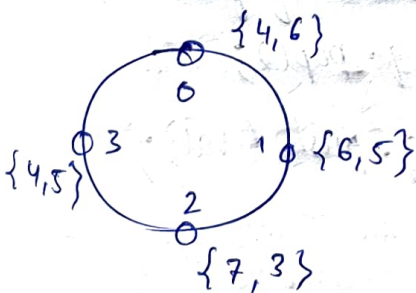
```
}
if (q.empty()) {
    ans.push_back("#");
}
```

```
return ans;
```

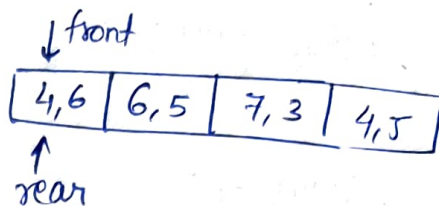
Q. Circular Tour.



i/p = 4 6 7 4 2
 6 5 3 5
 o/p = 1 (from where a truck
 can start & complete
 a whole circle).



approach



```
if (travel to next pump is possible)
```

```
rear++;
```

```
if (rear == front) {
    // cycle complete
    return front;
}
```

```
} else {
```

```
front = rear + 1;
rear = front;
```

code

```
int tour (petrolPump[], int n) {
```

```
    int deficit = 0;
```

```
    int balance = 0;
```

```
    int start = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        balance += P[i].petrol -
                    P[i].distance;
```

```
        if (balance < 0) {
```

```
            deficit += balance;
```

```
            start = i + 1;
```

```
            balance = 0;
```

```
        }
```

```
    }
```



```
else {
```

```
return start;
```

```
else {
```

return -1;

}

3

3;

logic

1	2				
6,5	7,6	4,7	10,8	6,6	5,4

Kami = 1

balance = 3

$$(\text{balancu} + \text{kami}) = \boxed{2 \geq 0}$$

ans possible

$$T_c = O(n)$$

$$SC = O(1)$$

ifp 1 2 | 3 4

O/P 1 3 2 4

approach

② push 1st half in stack

$$\begin{array}{|c|} \hline 2 \\ \hline 1 \\ \hline \end{array}$$

		3	4			
--	--	---	---	--	--	--

② stack $\rightarrow$ queue

3	4	2	1
---	---	---	---

③ Pop new 1st half & push back in queue

2	1	3	4
---	---	---	---

④ Again take 1st half in queue

1
2

314

⑤ Now since values in stack is in original order push 1 element from stack to queue & 1 element from queue to queue until stack becomes empty.

3	4	1	3	2	4
--------------	--------------	---	---	---	---

x
2

```
while (!s.empty()) {
```

```
int val = s.top();
```

```
s.pop();
```

2. push(val);

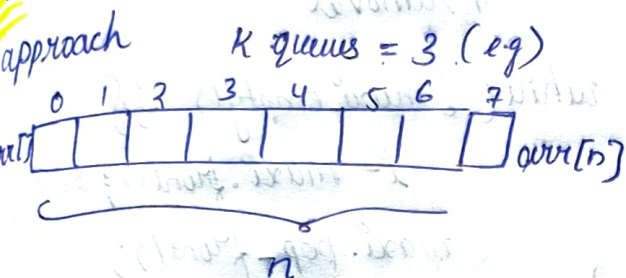
```
val = q.front();
```

q.pop();

2. push (val);

3

Q.) 'K' Queues in Array.



front[k] = [-1, -1, -1]

rear[k] = [-1, -1, -1]

next[n] = [1, 2, 3, 4, 5, 6, 7, -1]

freespots = 0

Push Operation

- Overflow check (freespots != -1)
- find index to insert index = freespots
- update freespots

freespots = next[freespots];

- if first element

if (front[qn] == -1)
front[qn] = index;

else

next[rear[qn]] = index;

next[index] = -1;

rear[qn] = index;

arr[index] = x;

Pop Operation

if (front[qn-1] == -1) {
cout << "underflow";
return;
}

// find index to pop

int index = front[qn-1];

// front aage badhao

front[qn-1] = next[index];

// update freespots

next[index] = freespots;

freespots = index;

} return arr[index];

Q.) Return Sum of min & max of window 'k'

eg { 2, 5, -1, 7, -3, 1, -2 }
k = 4

o/p { 6, 4, 4, 4 } = 18

approach 1 - Brute force

T.C = $O(n \cdot k)$

using queue for 'k' sized window

approach 2 deque used.

- ① deque $\langle \text{int} \rangle$ maxi(k) dec↓
- ② deque $\langle \text{int} \rangle$ mini(k) inc↑

① maxi.front() should be our max element in that window

② give min element ———

```
int solve(int *arr, int k) {  
    int n = arr.size();
```

```
    deque<int> maxi(k);
```

```
    deque<int> mini(k);
```

```
    for (int i=0; i<k; i++){
```

```
        while (arr[maxi.back()]  
                <= arr[i]) {
```

```
            maxi.pop-back();
```

```
        while (arr[mini.back()] >=  
                arr[i])
```

```
            mini.pop-back();
```

```
        maxi.push-back(i);
```

```
        mini.push-back(i);
```

```
    }
```

```
    int ans = 0;
```

```
    for (int i=k; i<n; i++){
```

```
        ans += arr[maxi.front()] +
```

```
            arr[mini.front()];
```

```
// next window
```

```
// removal
```

```
while (!maxi.empty() &&
```

```
    i - maxi.front() >= k)
```

```
    maxi.pop-front();
```

```
}
```

```
while (!mini.empty() &&
```

```
    i - mini.front() >= k)
```

```
    mini.pop-front();
```

```
}
```

```
// addition
```

```
while (!maxi.empty() &&
```

```
    arr[maxi.back()] <= arr[i])
```

```
    maxi.pop-back();
```

```
while (!mini.empty() &&
```

```
    arr[mini.back()] >= arr[i])
```

```
    mini.pop-back();
```

```
    maxi.push-back(i);
```

```
    mini.push-back(i);
```

```
}
```

```
ans += arr[maxi.front()] +
```

```
        arr[mini.front()];
```

```
}
```

```
return ans;
```

```
}
```